

PRÁCTICO N° 3

BASE DE DATOS II

ALUMNO: ABACA, JONATAN

PROFESOR: BARRIONUEVO, CESAR

AÑO: 2026

TRIGGERS

ESQUEMA DE TABLAS

```
CREATE TABLE clientes (  
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE,  
  saldo DECIMAL(12,2) DEFAULT 0.00,  
  activo TINYINT(1) DEFAULT 1,  
  fecha_registro DATETIME DEFAULT NOW() );
```

```
CREATE TABLE productos (  
  id_producto INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  precio DECIMAL(10,2) NOT NULL,  
  stock INT DEFAULT 0,  
  fecha_modificacion DATETIME );
```

```
CREATE TABLE ventas (  
  id_venta INT AUTO_INCREMENT PRIMARY KEY,  
  id_cliente INT NOT NULL,  
  id_producto INT NOT NULL,  
  cantidad INT NOT NULL,
```

```
total DECIMAL(12,2),
fecha DATETIME DEFAULT NOW(),
FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
FOREIGN KEY (id_producto) REFERENCES productos(id_producto) );

CREATE TABLE auditoria_precios (
id INT AUTO_INCREMENT PRIMARY KEY,
id_producto INT,
precio_anterior DECIMAL(10,2),
precio_nuevo DECIMAL(10,2),
usuario VARCHAR(100),
fecha_cambio DATETIME DEFAULT NOW() );
```

```
CREATE TABLE log_clientes_eliminados (
id INT AUTO_INCREMENT PRIMARY KEY,
id_cliente INT,
nombre VARCHAR(100),
email VARCHAR(150),
fecha_eliminacion DATETIME DEFAULT NOW() );
```

Ejercicio 1: AFTER INSERT - Descuento automatico de stock

Enunciado: Cada vez que se registre una venta, el stock del producto vendido debe reducirse automaticamente en la cantidad vendida.

```
DELIMITER $$ CREATE TRIGGER
trg_descontar_stock AFTER INSERT
ON ventas
FOR EACH ROW
BEGIN
UPDATE productos
```

```
SET stock = stock - NEW.cantidad

WHERE id_producto = NEW.id_producto;

END$$

DELIMITER ;

-- Prueba: INSERT INTO ventas (id_cliente, id_producto, cantidad, total) VALUES (1, 3,
5, 250.00);

SELECT stock FROM productos WHERE id_producto = 3;
```

Ejercicio 2: BEFORE INSERT - Calcular total automaticamente

Enunciado: Al insertar una venta, el campo total debe calcularse automaticamente multiplicando cantidad por el precio del producto.

```
DELIMITER $$

CREATE TRIGGER trg_calcular_total

BEFORE INSERT ON ventas

FOR EACH ROW

BEGIN

    DECLARE v_precio DECIMAL(10,2);

    SELECT precio INTO v_precio

    FROM productos

    WHERE id_producto = NEW.id_producto;

    SET NEW.total = NEW.cantidad * v_precio;

END$$

DELIMITER ;

-- Prueba:

INSERT INTO ventas (id_cliente, id_producto, cantidad) VALUES (1, 3, 5);
```

```
SELECT total FROM ventas ORDER BY id_venta DESC LIMIT 1;
```

Ejercicio 3: AFTER UPDATE - Auditoria de cambio de precios

Enunciado: Cuando el precio de un producto cambie, registrar automaticamente en auditoria_precios el precio anterior, el nuevo precio, el usuario MySQL y la fecha.

```
DELIMITER $$
```

```
CREATE TRIGGER trg_auditoria_precios
```

```
AFTER UPDATE ON productos
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF OLD.precio <> NEW.precio THEN
```

```
INSERT INTO auditoria_precios
```

```
(id_producto, precio_anterior, precio_nuevo, usuario, fecha_cambio)
```

```
VALUES
```

```
(NEW.id_producto, OLD.precio, NEW.precio, USER(), NOW());
```

```
END IF;
```

```
END$$
```

```
DELIMITER ;
```

```
-- Prueba:
```

```
UPDATE productos SET precio = 999.99 WHERE id_producto = 1;
```

```
SELECT * FROM auditoria_precios;
```

Ejercicio 4: BEFORE DELETE - Log antes de eliminar cliente

Enunciado: Antes de eliminar un cliente, guardar sus datos en log_clientes_eliminados para mantener un registro historico.

```
DELIMITER $$
```

```
CREATE TRIGGER trg_log_cliente
```

```
BEFORE DELETE ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO log_clientes_eliminados
        (id_cliente, nombre, email, fecha_eliminacion)
    VALUES
        (OLD.id_cliente, OLD.nombre, OLD.email, NOW());
END$$
DELIMITER ;
```

-- Prueba:

```
DELETE FROM clientes WHERE id_cliente = 5;
SELECT * FROM log_clientes_eliminados;
```

Ejercicio 5: BEFORE INSERT - Validar stock disponible

Enunciado: Antes de registrar una venta, verificar que exista suficiente stock. Si el stock es insuficiente, abortar la operacion con un mensaje de error.

Consejo: SIGNAL SQLSTATE '45000' es el codigo generico de error definido por el usuario en MySQL. Siempre incluye SET MESSAGE_TEXT para dar un mensaje descriptivo.

```
DELIMITER $$
CREATE TRIGGER trg_validar_stock
BEFORE INSERT ON ventas
FOR EACH ROW
BEGIN
    DECLARE v_stock INT;

    SELECT stock INTO v_stock
```

```

FROM productos

WHERE id_producto = NEW.id_producto;

IF v_stock < NEW.cantidad THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Stock insuficiente para completar la venta.';

END IF;

END$$

DELIMITER ;

-- Prueba (debe fallar si stock < cantidad):

INSERT INTO ventas (id_cliente, id_producto, cantidad) VALUES (1, 3, 9999);

```

Ejercicio 6: AFTER INSERT - Actualizar saldo del cliente

Enunciado: Cuando se registra una venta, descontar el total del saldo disponible del cliente automaticamente.

```

DELIMITER $$

CREATE TRIGGER trg_descontar_saldo

AFTER INSERT ON ventas

FOR EACH ROW

BEGIN

    UPDATE clientes

    SET saldo = saldo - NEW.total

    WHERE id_cliente = NEW.id_cliente;

END$$

DELIMITER ;

-- Prueba:

```

```
INSERT INTO ventas (id_cliente, id_producto, cantidad) VALUES (1, 2, 3);
```

```
SELECT saldo FROM clientes WHERE id_cliente = 1;
```

Ejercicio 7: BEFORE UPDATE - Proteger email, normalizar a minúsculas

Enunciado: Antes de actualizar el email de un cliente, normalizar a minúsculas y verificar que no exista otro cliente con ese email.

```
DELIMITER $$
```

```
CREATE TRIGGER trg_normalizar_email
```

```
BEFORE UPDATE ON clientes
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE v_count INT;
```

```
    SET NEW.email = LOWER(NEW.email);
```

```
    SELECT COUNT(*) INTO v_count
```

```
    FROM clientes
```

```
    WHERE email = NEW.email
```

```
    AND id_cliente <> OLD.id_cliente;
```

```
    IF v_count > 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'El email ya está registrado por otro cliente.';
```

```
    END IF;
```

```
END$$
```

```
DELIMITER ;
```

```
-- Prueba:
```

```
UPDATE clientes SET email = 'USUARIO@EJEMPLO.COM' WHERE id_cliente = 1;  
SELECT email FROM clientes WHERE id_cliente = 1;
```

Ejercicio 8: AFTER DELETE - Restaurar stock al cancelar venta

Enunciado: Cuando se elimine una venta (cancelacion), devolver automaticamente la cantidad al stock del producto correspondiente.

```
DELIMITER $$
```

```
CREATE TRIGGER trg_restaurar_stock
```

```
AFTER DELETE ON ventas
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    UPDATE productos
```

```
    SET stock = stock + OLD.cantidad
```

```
    WHERE id_producto = OLD.id_producto;
```

```
END$$
```

```
DELIMITER ;
```

-- Prueba:

```
DELETE FROM ventas WHERE id_venta = 7;
```

```
SELECT stock FROM productos WHERE id_producto = OLD.id_producto;
```